Introduction to Machine Learning (67577)

Exercise 2
Regression, Classification & PAC

Second Semester, 2026

Contents

1 Theoretical Part

Based on Lectures 2,3 and Recitations 3,4

1.1 Least Squares

Given a sample $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, the ERM rule for linear regression w.r.t. the squared loss is

$$\hat{\mathbf{w}} \in \underset{\mathbf{w}\in\mathbb{R}^d}{\operatorname{argmin}} \ ||\mathbf{X}\mathbf{w} - \mathbf{y}||^2$$

where $\mathbf{X}$ is the input matrix of the linear regression with rows as samples and y the vector of responses. Let $\mathbf{X} = U\Sigma V^\top$ be the SVD of $\mathbf{X}$, where U is a $m \times m$ orthonormal matrix, Σ is a $m \times d$ diagonal matrix, and V is an $d \times d$ orthonormal matrix. Let $\sigma_i = \Sigma_{i,i}$ and note that only the non-zero σ_i-s are singular values of $\mathbf{X}$. Recall that the pseudoinverse of $\mathbf{X}$ is defined by $\mathbf{X}^\dagger = V\Sigma^\dagger U^\top$ where $\Sigma^\dagger$ is an $d \times m$ diagonal matrix, such that

$$\Sigma^\dagger_{i,i} = \begin{cases} \sigma_i^{-1} & \sigma_i \neq 0 \\ 0 & \sigma_i = 0 \end{cases}$$

1. Assuming that $\mathbf{X}^\top\mathbf{X}$ is invertible, show that the general solution we derived in recitation 3 $\left(\mathbf{X}^\dagger\mathbf{y}\right)$ equals to the solution you have seen in lecture 1 $\left(\left[\mathbf{X}^\top\mathbf{X}\right]^{-1}\mathbf{X}^\top\mathbf{y}\right)$.

1.2 Hard- & Soft-SVM

In class we saw the Hard-SVM classification model.

1. Prove that the following Hard-SVM optimization problem is a Quadratic Programming problem:

$$\underset{(\mathbf{w},b)}{\operatorname{argmin}} ||\mathbf{w}||^2 \quad \text{s.t.} \quad \forall i \ y_i\left(\langle \mathbf{w}, \mathbf{x}_i\rangle + b\right) \geq 1 \tag{1}$$

That is, find matrices Q and A and vectors $\mathbf{a}$ and $\mathbf{d}$ such that the above problem can be written in the following format

$$\underset{\mathbf{v}\in\mathbb{R}^n}{\operatorname{argmin}} \tfrac{1}{2}\mathbf{v}^\top Q\mathbf{v} + \mathbf{a}^\top\mathbf{v} \quad \text{s.t.} \quad A\mathbf{v} \leq \mathbf{d} \tag{2}$$

Hint: Observe that $||\mathbf{w}||^2 = \mathbf{w}^\top\mathbf{I}\mathbf{w}$

1.3 VC-Dimension

1. Let $\mathcal{X} = \{0,1\}^d$ and $\mathcal{Y} = \{0,1\}$, and assume $d \geq 2$. Each sample $(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$ consists of an assignment to d boolean variables ($\mathbf{x}$) and a label (y). For each boolean variable x_k, $k \in [d]$, there are two literals: x_k and $\bar{x}_k = 1 - x_k$. The class $\mathcal{H}_{\text{con}}$ is defined by boolean conjunctions over any subset of these $2d$ literals. Compute the VC dimension of $\mathcal{H}_{\text{con}}$ and prove your answer.

Given an integer k, let $([a_i, b_i])_{i=1}^k$ be any set of k intervals on $\mathbb{R}$ and define their union $A = \cup_{i=1}^k [a_i, b_i]$. The hypothesis class $\mathcal{H}_{k-intervals}$ includes the functions: $h_A(x) := \mathbb{1}_{[x\in A]}$, for all choices of k intervals. Find the VC-dimension of $\mathcal{H}_{k-intervals}$ and prove your answer. Show that if we let A be any finite union of intervals (i.e. k is unlimited), then the resulting class $\mathcal{H}_{intervals}$ has VC-dimension ∞.

2 Practical Part - Regression

R In this section, you'll work on implementing code that may be new to many of you. It's not expected that you know these concepts beforehand; part of this exercise is learning as you go.

2.1 Fitting A Linear Regression Model

In this question you will have to deal with a real-world dataset and fit a linear regression model to it. As data is noisy, messy and difficult, take the time to "play" and get familiar with it. The data is provided in `house_prices.csv`.

Implement the following:

1. In the `linear_regression.py` file, we provide a template for the `LinearRegression` class. Follow the provided documentation, and implement `LinearRegression` class API according to the class and methods documentation.

Then, implement code of following questions in the `house_price_prediction.py` file:

2. Randomly split the data frame to a training set (75%) and test set (25%). To make sure your training data does not change every time you re-run the code, set a `random_seed` or `random_state` parameter.
3. Implement the `preprocess_train` function. The function receives a loaded `pandas.DataFrame` object of the features matrix, and a `pandas.Series` object of the respone vector, and returns them after preprocessing. Make sure that if you remove a sample, you remove it properly from both the observation matrix as well as from the response vector.
   Explore the data (some information can be found on Kaggle) and perform any necessary preprocessing (such as but not limited to):
   - What sort of values are valid for different types of features? Can the year that the house was built be later than the year that the house was renovated? Can a living room size be too small?
   - Are there any additional features that might be beneficial for predicting the house price and that can be derived from existing features?

   R Why do we want the split before preprocessing? We aim to construct a model capable of predicting on new samples it has not encountered previously, a concept known as generalization. To evaluate the success of our model, we require the test set to remain concealed and independent from our design choices. If we were to include the test set in preprocessing tasks, such as exploring feature correlations with the response, we would risk contaminating our training process. Consequently, our assessments of our model's generalization performance would be compromised, leading to inaccurate conclusions.

4. Implement the `preprocess_test` function. This function receives as an input only the features matrix of the *test set*, and outputs a pre-processed features matrix that corresponds with what you implemented in the `preprocess_train`. For example, if you added an additional column that indicates if the house is recently renovated, you need to add this column also in the test set, or the coefficients that you fit will not match. Note: You are not allowed

to remove samples (i.e., entire rows) from the test set, but only add/remove columns or edit values in specific cells.

> Why are we not allowed to remove rows from the test set? Because doing so would invalidate the reported results as they would not accurately reflect the performance on the test set. For instance, if you were asked to predict on 100 samples and chose to remove 50 of them for any reason, the accuracy reported would not provide a comprehensive understanding of how your model operates on real-world, non-hand-crafted data.

5. Fit a linear regression model over increasing percentages of the *training set*, and measure the loss over the *test set*:
   - Iterate for every percentage $p = 10\%, 11\%, ..., 100\%$ of the training set.
   - Sample $p\%$ of the train set. You can use the `pandas.DataFrame.sample` function.
   - Repeat sampling, fitting and evaluating 10 times for each value of p.

   Plot the mean loss as a function of $p\%$, as well as a confidence interval of $mean(loss) \pm 2 * std(loss)$. If implementing using the `Plotly` library, see how to create the confidence interval in Chapter 2 - Linear Regression code examples.

   Add the plot to the `Answers.pdf` file and explain what is seen. Address both trends in loss and in confidence interval as function of training size.

   > In this question, our objective is to assess the impact of enlarging the training set on the accuracy of the test set. By iteratively sampling the data 10 times, we aim to enhance the reliability of our conclusions. A single sampling instance may inadequately capture the data's variability, potentially leading to skewed results. However, by conducting the sampling procedure repeatedly, we mitigate the influence of outlier samples and ensure a more reliable conclusions regarding our model.

   Implementation clarification:
   (a) Notice that when predicting on the training set, you need to be consistent with some of the preprocessing

2.2 Polynomial Fitting

In this question you are going to use linear regression model, for a problem that at first sight does not seem like linear regression. But, after input transformation, linear regression becomes applicable.

Polynomial fitting is a specialized application of linear regression, aimed at solving the task of identifying a polynomial of degree k that accurately models a given set of data.
In mathematical terms, this involves finding a function $f(x) = y$, where both x and y are real numbers ($x, y \in \mathbb{R}$), and f is a polynomial of degree k.
Here comes the "trick": This problem can also be reformulated in terms of vectors. Instead of our input to be $x \in \mathbb{R}$, we represent our input by the following vector $\hat{x} = [x^0, x^1, ..., x^k]$, so that the $i'th$ entry is x to the power of i. So now our input $\hat{x}$ is in $\mathbb{R}^k$, and polynomial fitting then becomes the task of finding the coefficients $w = [w_0, w_1, ..., w_k]$ so that the dot product $w^T \cdot \hat{x} = y$ is a different representation for the original $f(x) = y$.
This reformulation transforms the problem into a classic linear regression challenge, where the goal is to find the weight vector w that best maps the transformed input vector to the output y.

Class implementation:

1. Implement the `PolynomialFitting` class in the `polynomial_fitting.py` file as specified in class documentation. Avoid repeating code from the `LinearRegression` class and instead use inheritance or composition patterns. You are allowed to use the `np.vander` function.

In the following questions you will use the Daily Temperature of Major Cities dataset. Notice, that the supplied file `city_temperature.csv` is a modified subset of the dataset found on Kaggle containing only 4 countries. You will fit and analyse performance of a polynomial model over the dataset.

2. Implement the `load_data` function in the `city_tempretaure_prediction.py` file.
   - When loading the dataset remember to deal with invalid data.
   - Use the `parse_dates` argument of the `pandas.read_csv` to set the type of the `'Date'` column.
   - Add a `'DayOfYear'` column based on the `'Date'` column. This column will be the feature to be used for the polynomial fitting.

3. Filter the dataset to contain samples only from the country of Israel. Investigate how the average daily temperature (`'Temp'` column) change as a function of the `'DayOfYear'`.
   - Plot a scatter plot showing this relation, and color code the dots by the different years (make sure color scale is discrete and not continuous). What polynomial degree might be suitable for this data?
   - Group the samples by `'Month'` (have a look at the pandas `'groupby'` and `'agg'` functions) and plot a bar plot showing for each month the standard deviation of the daily temperatures. Suppose you fit a polynomial model (with the correct degree) over data sampled uniformly at random from this dataset, and then use it to predict temperatures from random days across the year. Based on this graph, do you expect a model to succeed equally over all months or are there times of the year where it will perform better than on others? Explain your answer.

   Add plots and answers to the `Answers.pdf` file.

Over the subset containing observations (i.e., samples) only from Israel perform the following:

- Randomly split the dataset into a training set (75%) and test set (25%).
- For every value $k \in [1, 10]$, fit a polynomial model of degree k using the training set.
- Record the loss of the model over the test set, rounded to 2 decimal places.

Print the test error recorded for each value of k. In addition plot a bar plot showing the test error recorded for each value of k. Based on these which value of k best fits the data? In the case of multiple values of k achieving the same loss select the simplest model of them. Are there any other values that could be considered?

3 Practical Part - Classification

In this exercise, you will explore and visualize various classification methods. The answers and plots will be in submitted in `Answers.pdf` file, and the code will be submitted in `classifiers_comparison.py`.

Implementation details:

- You may use the packages `scikit-learn`, `matplotlib`, `numpy`, `seaborn` (Optional). No other packages are allowed.
- The implementation should be done under the main script found in the `classifiers_comparison.py` file.
- Note that we have added a `utils.py` for your convenience, using it is optional.

3.1 Classification Boundaries and Model Comparison

Data Generation: Generate 200 samples of the following datasets:

- **Moons:** Generate moons dataset using `scikit-learn`'s function `make_moons`. Set `noise=0.2`.
- **Circles**: Generate circle dataset using `scikit-learn`'s function `make_circles`. Set `noise=0.1`.
- **Two Guassians**: Generate two Gaussian distributions with means $(-1,-1),(1,1)$ and covariance matrix
$$\begin{bmatrix} 0.5 & 0.2 \\ 0.2 & 0.5 \end{bmatrix}$$
- Split the datasets to train (0.8) and test (0.2)

Classifiers: Apply the following classifiers over these train datasets:

1. SVM with $\lambda = 5$
2. Decision Tree with depth 7
3. KNN with $k = 5$

Boundary Plots: Visualize the data points, using distinct colors to represent different labels. Additionally, display the decision boundary of a classifier by coloring the background. For each coordinate on the plot, color it according to the class the classifier would predict, using slightly lighter shades of the corresponding data point colors.

Questions:

1. For each classifier and each dataset, provide a decision boundary plot. Include the accuracy over the test dataset and model in the title of each plot.
2. Write a comparison of how the classifiers perform on different types of data distributions, and explain why each classifier is or is not a good fit for the dataset.
3. Explain how each algorithm shapes the decision boundary.